Duração 60 minutos	Introdução às Redes de Comunicação 2024/2025 - Engenharia Informática	Teste UDP P5 – 31/10/2024 14h30 – AGP – DEC

Este teste é sem consulta dos apontamentos pessoais de cada aluno (escritos, ou imprimidos, em papel). No decorrer do teste não é permitido partilhar quaisquer documentos ou objetos. No decorrer do teste não é permitido, ao aluno, a utilização de auscultadores, de auriculares, de telemóveis, de *smartwatches*, de *tablets* ou de computadores.

Coloque o nome completo e o número de aluno: (1) na folha da prova (2) na folha de rascunho. Não risque a ficha de consulta autorizada para o teste.

No fim, antes de entregar a prova, coloque dentro da folha da prova: (1) o enunciado do teste; (2) a folha de rascunho; (3) a ficha de consulta autorizada para o teste.

Uma empresa que gere um sistema de reciclagem de papel, recolhido em diversos ecopontos, tem um conjunto de pequenas viaturas elétricas que efetuam a recolha do papel colocado nos diversos ecopontos. Para melhorar o desempenho na recolha do papel que está nos ecopontos, a empresa pretende implementar um sistema cliente-servidor (UDP), entre os ecopontos, um gestor das recolhas (GdR) e cada uma das viaturas elétricas de recolha (VdR).

Quando um ecoponto está quase cheio, o ecoponto envia um «alerta» ao gestor das recolhas. Com base nessa mensagem, o GdR envia um pedido, com o número de identificação do ecoponto (por exemplo, 10101), para uma viatura de recolha (VdR) para que, essa viatura, vá recolher o papel que está no ecoponto (10101).

Quando uma VdR recebe um pedido de recolha, a VdR verifica se o pedido vem do GdR. Depois, a viatura responde ao GdR, enviando o seu número de identificação (e fica à espera de outro pedido do GdR). Depois de receber esta resposta, o GdR pode iniciar outro pedido (para qualquer uma das viaturas).

Os valores para inicializar uma viatura de recolha são passados através da linha de comando da aplicação VdR. Por exemplo: `VdR 10.20.30.2 7001 711 10.20.30.1 6000.` Os elementos da linha de comando correspondem ao IP usado pela viatura (10.20.30.2), ao porto usado pela viatura (7001), ao número de identificação da viatura (711), ao IP (10.20.30.1) e ao porto (6000) usados pelo GdR.

- Implemente, em C, a aplicação «viatura de recolha», completando o código do `VdR.c,` nas zonas indicadas (de [A] a [J]).
- Na implementação desta aplicação (VdR), considere, para as diversas aplicações existentes neste sistema cliente-servidor, a possibilidade de as aplicações estarem a ser utilizadas em ambientes distribuídos heterogéneos.
- Cotação de cada uma das alíneas – 10%.

VdR.c

```
int main(int argc, char *argv[])
{

if (argc != 6) {
       fprintf(stderr, "\n Sintaxe errada -> %s \n", argv[0]);
       char c = getchar();
       exit(EXIT_FAILURE);
       }

int IdV= atoi(argv[3]); //identidade da viatura

WSADATA wsaData;
int iResult = WSAStartup(MAKEWORD(2,2), &wsaData);
if (iResult != 0) {
       fprintf(stdout, "\n <VdR-%d> WSAStartup failed: %d\n", IdV, iResult);
       char c= getchar();
       exit(1);
       }


SOCKET socket_viatura= [A]

if(socket_viatura  == INVALID_SOCKET)
       Abort("Impossibilidade de abrir o socket", socket_viatura, IdV);

struct sockaddr_in viatura_addr;

memset((char*)&viatura_addr, 0, sizeof(viatura_addr));
viatura_addr.sin_family = AF_INET;
viatura_addr.sin_addr.s_addr = [B1]
viatura_addr.sin_port = [B2]

int nbytes= [C] //associa o socket ao endereço

if(nbytes == SOCKET_ERROR)
       Abort("Falha ao associar um IP + porto", socket_viatura, IdV);


struct sockaddr_in gdr_addr;
int pedido;

while(1) {

    fprintf(stdout,"\n <VdR-%d> Espera por um pedido...\n", IdV);
    int tam_gdr_addr = sizeof(gdr_addr);

    int nbytes= [D] //aguarda pelo pedido do GdR
    if(nbytes == SOCKET_ERROR)
       Abort("Erro na recepcao do pedido", socket_viatura, IdV);

    pedido = [E] //identidade do eco-ponto
    fprintf(stdout,"\n <VdR-%d> Pedido recebido {%d} do GdR", IdV, pedido);

    //verifica a origem do pedido

    //porto do pedido
    unsigned int porto_pedido= [F]

    //IP do pedido
    char ip_pedido[IPSIZE];
    strcpy_s(ip_pedido,IPSIZE,(char*)[G]);

    if([H])
         fprintf(stdout,"\n A mensagem veio do GdR);
    else {fprintf(stdout,"\n A mensagem nao veio do GdR); continue;}
```

```
    //Define a resposta ao gestor

    int resposta = [I] //identidade da viatura
    nbytes = [J]

    if(nbytes == SOCKET_ERROR)
       fprintf(stdout, "\n <VdR-%d> Erro ao enviar a resposta do pedido ao GdR", IdV);
    else fprintf(stdout,"\n <VdR-%d> Resposta do pedido enviada ao GdR”, IdV);

    }

closesocket(socket_viatura);
WSACleanup();
exit(EXIT_SUCCESS);
}
```

VdR.h

```
#include <stdio.h>
#include <string.h>
#include <winsock.h>

#define BUFFERSIZE 4096
#define IPSIZE 20

void Abort(char* msg, SOCKET s, int id);
```

1. [A] socket(PF_INET, SOCK_DGRAM, 0);
2. [B1] inet_addr(argv[1]);
2. [B2] htons(atoi(argv[2]));
3. [C] bind(socket_viatura, (struct sockaddr*)&viatura_addr, sizeof(viatura_addr));
4. [D] recvfrom(socket_viatura, (char*)&pedido, sizeof(pedido), 0,
   (struct sockaddr *)&gdr_addr , &tam_gdr_addr);
5. [E] ntohs(pedido);
6. [F] ntohs(gdr_addr.sin_port);
7. [G] inet_ntoa(viatura_addr.sin_addr));
8. [H] strcmp(ip_pedido, argv[4]) == 0 && porto_pedido == atoi(argv[5])
9. [I] htons(IdV);
10. [J] sendto(socket_viatura, (char*)&resposta, (int)sizeof(resposta), 0,
    (struct sockaddr*)&gdr_addr, sizeof(gdr_addr));